

# Guía Universal de Integración de Juegos - Gamispace

*Gamispace* es una plataforma educativa que integra asignaturas, usuarios y videojuegos. El sistema está diseñado para ser "Plug & Play" y es completamente agnóstico al motor. La plataforma se encarga del trabajo pesado (seguridad, bases de datos y persistencia). Tu objetivo es simplemente conectar tu juego a nuestro puente de comunicación.

**⚠ Nota importante sobre Copiar y Pegar:** Si al copiar los bloques de código desde este documento experimentas errores de compilación, es muy probable que tu sistema operativo o portapapeles esté introduciendo saltos de línea ocultos. Te recomendamos **pegar el código primero en un editor de texto plano** (como el Bloc de Notas) antes de llevarlo a tu motor, o utilizar los archivos de código fuente directamente proporcionados junto a esta guía.

## 1. Entorno de Pruebas Local (Mock API)

Una vez hayas implementado lo descrito a continuación puedes probar que tu juego envía y recibe datos sin tener que compilar y subir el juego a la plataforma en cada intento, puedes simular el servidor directamente en tu navegador. Abre tu juego exportado en el navegador, pulsa **F12** para abrir la consola, pega este código y pulsa Enter:

```
window.GamispaceAPI = {
  requestProgress: function() {
    console.log("[Mock] Petición de progreso recibida.");
    setTimeout(() => {
      if (this.onProgressReceived) {
        this.onProgressReceived(["level": 1, "score": 100, "time": 10.5, "completed":
true}]);
      }
    }, 500);
  },
  sendPlayData: function(level, score, time, completed) {
    console.log("[Mock] Guardando -> Nivel: " + level + " | Puntos: " + score + " | Tiempo: "
+ time + " | Completado: " + completed);
    setTimeout(() => {
      if (this.onPlaySaved) {
        this.onPlaySaved(true);
      }
    })
  }
}
```

```

    }, 500);
  }
};

```

## 2. Guía Práctica para Unity (WebGL)

### Paso 1: El Traductor (.jslib)

Crea un archivo llamado GamispacPlugin.jslib en la ruta de tu proyecto Assets/Plugins/WebGL/ y pega el siguiente código:

```

mergeInto(LibraryManager.library, {
  IsGamispacAPIReadyJS: function() {
    return (typeof window.GamispacAPI !== 'undefined');
  },
  RequestProgressJS: function () {
    if (window.GamispacAPI) {
      window.GamispacAPI.onProgressReceived = function(json) {
        SendMessage('WebBridge', 'OnProgressReceived', json);
      };
      window.GamispacAPI.requestProgress();
    }
  },
  SendPlayDataJS: function (level, score, time, completedStatus) {
    if (window.GamispacAPI) {
      window.GamispacAPI.onPlaySaved = function(success) {
        SendMessage('WebBridge', 'OnPlaySaved', success ? 1 : 0);
      };
      window.GamispacAPI.sendPlayData(level, score, time, completedStatus === 1);
    }
  }
});

```

### Paso 2: El Gestor C# robusto (WebBridge.cs)

Crea un GameObject vacío en tu primera escena, nómbralo **EXACTAMENTE** WebBridge y añádele este script. Este código cuenta con protección ante la optimización de código de Unity (IL2CPP) y compatibilidad total con el Editor local:

```

using UnityEngine;
using System.Collections;
using System.Runtime.InteropServices;
using System.Collections.Generic;
using UnityEngine.Scripting; // Requerido para impedir que el optimizador borre código

[Preserve] // Evita que el optimizador IL2CPP elimine la clase completa al compilar
public class WebBridge : MonoBehaviour
{
    public static WebBridge instance;

    [DllImport("__Internal")] private static extern bool IsGamispaceAPIReadyJS();
    [DllImport("__Internal")] private static extern void RequestProgressJS();
    [DllImport("__Internal")] private static extern void SendPlayDataJS(int level, int score,
float time, int completedStatus);

    public Dictionary<int, NivelData> datosProgreso = new Dictionary<int, NivelData>();
    public System.Action OnDatosActualizados;

    void Awake()
    {
        if (instance == null) { instance = this; DontDestroyOnLoad(gameObject);
gameObject.name = "WebBridge"; }
        else { Destroy(gameObject); }
    }

    IEnumerator Start()
    {
        int intentos = 0;
        #if UNITY_WEBGL && !UNITY_EDITOR
            // Polling: Esperamos hasta que la plataforma haya inyectado la API de forma segura
            while (!IsGamispaceAPIReadyJS() && intentos < 50)
            {
                yield return new WaitForSeconds(0.1f);
                intentos++;
            }
            RequestProgressJS();
        #else

```

```

        // Mantiene el iterador activo en el Editor local, evitando el error de compilación
        CS0161
        yield return null;
        Debug.Log("[Simulación Editor] Modo local activo. Puente listo.");
    #endif
}

```

```

public void GuardarPartida(int nivel, int puntos, float tiempo, bool nivelSuperado)
{
    #if UNITY_WEBGL && !UNITY_EDITOR
        SendPlayDataJS(nivel, puntos, tiempo, nivelSuperado ? 1 : 0);
    #else
        Debug.Log($"[Simulación] Nivel {nivel} guardado. Puntos: {puntos} | Tiempo:
{tiempo}s | Estado: {nivelSuperado}");
    #endif
}

```

[Preserve] // Evita que IL2CPP elimine este método invocado externamente desde JS mediante SendMessage

```

public void OnProgressReceived(string jsonString)
{
    // Validación temprana y segura para manejar limpiamente a jugadores
    completamente nuevos
    if (jsonString == "[]" || string.IsNullOrEmpty(jsonString)) {
        OnDatosActualizados?.Invoke(); return; }
}

```

```

try
{
    NivelData[] progreso = JsonHelper.FromJson<NivelData>(jsonString);
    datosProgreso.Clear();
    foreach (var dato in progreso) { datosProgreso[dato.level] = dato; }
    OnDatosActualizados?.Invoke();
}
catch (System.Exception e)
{
    Debug.LogError("Error al procesar el JSON de progreso web: " + e.Message);
}
}

```

```

[Preserve] // Evita que IL2CPP elimine este método invocado desde JS
public void OnPlaySaved(int successStatus)
{
    if (successStatus == 1) { RequestProgressJS(); }
    else { Debug.LogError("Error de red: La plataforma web no pudo procesar el
guardado."); }
}
}

```

```

[System.Serializable]
[Preserve] // Conserva la estructura de datos intacta para la serialización JSON en WebGL
public class NivelData { public int level; public int score; public float time; public bool
completed; }

```

```

[Preserve]
public static class JsonHelper {
    public static T[] FromJson<T>(string json) {
        return JsonUtility.FromJson<Wrapper<T>>("{ \"array\": " + json + " }").array;
    }
    [System.Serializable] [Preserve] private class Wrapper<T> { public T[] array; }
}

```

## Paso 3: Ejemplos Prácticos de Implementación

### Ejemplo 1: Registrar el fin de un nivel (GameManager)

```

public class GameManager : MonoBehaviour
{
    public int nivelActual = 1;
    private float tiempoJugado = 0f;

    void Update() { tiempoJugado += Time.deltaTime; }

    public void LlegarALaMeta(int puntosConseguidos)
    {
        WebBridge.instance.GuardarPartida(nivelActual, puntosConseguidos, tiempoJugado,
true);
    }
}

```

```
}
```

## Ejemplo 2: Recuperar y mostrar récords en el Menú de Selección

```
using UnityEngine;
using TMPro;

public class BotonMenuNivel : MonoBehaviour
{
    public int nivelRepresentado = 1;
    public TMP_Text textoRécord;

    void Start()
    {
        if (WebBridge.instance.datosProgreso.ContainsKey(nivelRepresentado))
        {
            int puntos = WebBridge.instance.datosProgreso[nivelRepresentado].score;
            textoRécord.text = "Récord: " + puntos;
        }
        else
        {
            textoRécord.text = "Sin jugar";
        }
    }
}
```

## 3. Guía Práctica para Godot 4 (Web)

### Paso 1: El Script Autoload (Gestor)

Crea un script llamado WebBridge.gd y configúralo como **Autoload (Singleton)** en la configuración de tu proyecto:

```
extends Node

var datos_progreso = {}

var cb_progress = JavaScriptBridge.create_callback(_recibir_progreso)
var cb_saved = JavaScriptBridge.create_callback(_confirmar_guardado)
```

```
func _ready():  
    if OS.has_feature("web"):  
        var intentos = 0  
        var api = JavaScriptBridge.get_interface("GamispaceAPI")
```

```
        while api == null and intentos < 50:  
            await get_tree().create_timer(0.1).timeout  
            api = JavaScriptBridge.get_interface("GamispaceAPI")  
            intentos += 1
```

```
        if api:  
            api.onProgressReceived = cb_progress  
            api.onPlaySaved = cb_saved  
            api.requestProgress()
```

```
func guardar_partida(nivel: int, puntos: int, tiempo: float, completado: bool):
```

```
    if OS.has_feature("web"):  
        var api = JavaScriptBridge.get_interface("GamispaceAPI")  
        if api:  
            api.sendPlayData(nivel, puntos, tiempo, completado)  
        else:  
            print("[Simulación] Nivel guardado.")
```

```
func _recibir_progreso(args):
```

```
    var json = args[0]  
    if json == "[]" or json == "": return
```

```
    var progreso = JSON.parse_string(json)  
    if typeof(progreso) != TYPE_ARRAY:  
        printerr("Error: Formato JSON inesperado")  
    return
```

```
    datos_progreso.clear()  
    for dato in progreso:  
        datos_progreso[dato.level] = dato
```

```
func _confirmar_guardado(args):
```

```
    if args[0]:
```

```
JavaScriptBridge.get_interface("GamispaceAPI").requestProgress()  
else:  
    printerr("Error de red: La partida no se pudo guardar.")
```

## Paso 2: Ejemplos Prácticos de Implementación (Godot)

### Ejemplo 1: Terminar un nivel (Colisión en meta)

```
extends Area2D  
  
func _on_body_entered(body):  
    if body.name == "Player":  
        # Nivel 1, 500 puntos, 45.2s, true (Superado)  
        WebBridge.guardar_partida(1, 500, 45.2, true)
```

### Ejemplo 2: Recuperar y mostrar récords en un Menú (Label)

```
extends Label  
  
@export var nivel_representado: int = 1  
  
func _ready():  
    # Esperamos un frame para asegurar que el Autoload (WebBridge) ya ha inicializado  
    await get_tree().process_frame  
  
    # Comprobamos si el diccionario tiene guardado este nivel  
    if WebBridge.datos_progreso.has(nivel_representado):  
        var puntos = WebBridge.datos_progreso[nivel_representado].score  
        text = "Récord: " + str(puntos)  
    else:  
        text = "Sin jugar"
```

## 4. Otros Motores (Phaser, Construct, GameMaker)

Gamispace es 100% compatible con cualquier software capaz de exportar a HTML5 (Web). Debido a que los despliegues web ejecutan código JavaScript de manera nativa en el navegador, el flujo consiste únicamente en invocar y escuchar el objeto global de la ventana: `window.GamispaceAPI`.

**Para enviar datos:** Llama de manera directa a la API usando la capa de integración de



JavaScript de tu motor:

- **Phaser:** `window.GamispacAPI.sendPlayData(1, 500, 15.2, true);`
- **Construct 3:** Añade el objeto de sistema *Browser* y usa la acción *Execute Javascript* para invocar la misma instrucción.
- **GameMaker Studio:** Declara una extensión web JS simple con una función puente que llame al objeto nativo.

**Para recibir datos (Con Polling de seguridad):** Implementa este código recursivo de conexión en tu inicio. Esto asegura que tu motor espere a que la plataforma de Gamispac esté completamente lista antes de pedir datos:

```
// Función recursiva segura para conectar con la plataforma
function iniciarConexionGamispac() {
  if (typeof window.GamispacAPI !== 'undefined') {
    // 1. Asignamos el receptor
    window.GamispacAPI.onProgressReceived = function(jsonString) {
      if (jsonString === "[]" || jsonString === "") return;
      var progreso = JSON.parse(jsonString);

      // Aquí conectas el array 'progreso' con el código de tu juego
      console.log("Datos recibidos correctamente.");
    };
    // 2. Pedimos los datos
    window.GamispacAPI.requestProgress();
  } else {
    // Si la API aún no está lista, reintentamos en 100 milisegundos
    setTimeout(iniciarConexionGamispac, 100);
  }
}

// Arrancamos el proceso al inicio del juego
iniciarConexionGamispac();
```

## 5. Reglas de Exportación y Subida (.zip)

Para asegurar un despliegue exitoso e instantáneo en Gamispac, tu compresión final debe cumplir estrictamente con los siguientes requisitos:

1. Genera tu compilación exclusivamente bajo el formato **WebGL / HTML5**. Se recomienda activar la compresión **Brotli** o **Gzip** en los perfiles de publicación para acelerar la

velocidad de carga inicial en la web.

2. **Ubicación del archivo raíz:** Al comprimir el archivo .zip para subirlo a la plataforma, el archivo básico index.html debe estar situado exactamente en el directorio raíz del paquete comprimido. No incluyas una carpeta contenedora principal antes del archivo index. Para Unity basta con comprimir a .zip la carpeta que seleccione en build settings, la que tiene el nombre del proyecto y contiene el index.html y las carpetas Build y TemplateData.
3. **Inyección automática:** No realices modificaciones manuales sobre el archivo index.html generado. Gamispace inyecta de manera transparente y dinámica el entorno completo de la API durante la carga del juego en la plataforma.

## 6. Solución de Problemas (Troubleshooting)

| Problema                                                                                                        | Causa y Solución                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unity: El juego colapsa por completo en el navegador mostrando el error genérico "RuntimeError: null function". | <p><b>1. Code Stripping (IL2CPP):</b> El optimizador automático eliminó tus funciones por considerarlas código muerto. Asegúrate de añadir el decorador de metadatos [UnityEngine.Scripting.Preserve] encima del script, clases de datos y métodos invocados por strings en JavaScript.</p> <p><b>2. Excepciones Ocultas en WebGL:</b> Por defecto, Unity silencia los fallos de C# en web. Para depurar el error real de tu lógica, ve en el menú de Unity a Edit &gt; Project Settings &gt; Player &gt; WebGL &gt; Publishing Settings y cambia el parámetro <b>Enable Exceptions</b> al modo <b>Explicit</b>. Esto expondrá la traza real (ej. <i>NullReferenceException</i>) en la consola web de tu navegador.</p> |
| Unity: El sistema SendMessage lanza fallos                                                                      | Verifica meticulosamente que exista un GameObject en tu primera escena                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| Problema                                                                                 | Causa y Solución                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| explícitos o silenciosos.                                                                | jerárquica activa, nombrado con precisión ortográfica como <b>WebBridge</b> (sensible a mayúsculas y minúsculas) y que no se encuentre anidado como hijo de ningún otro nodo estructural.                                                                                                      |
| Unity: Fallo de compilación local inmediato al importar el script de la guía.            | Ocurre al compilar la corrutina vacía fuera de la plataforma web. Asegúrate de que el bloque condicional del preprocesador cuente con su sentencia de escape nativa <code>#else yield return null; #endif</code> para satisfacer los requisitos sintácticos del compilador de C# en el Editor. |
| El juego se congela o no actualiza el estado de las variables tras efectuar un guardado. | Inspecciona la consola del navegador mediante <b>F12</b> . Si la función de retorno del guardado notifica un estado nulo o falso, valida el estado de la conexión HTTP del cliente, la persistencia de la sesión del usuario o posibles interferencias de red en el canal de comunicación.     |